

PROBLEMATIC:

Learning is often more effective when people explain concepts to each other, exchange ideas, and solve problems together. However, peer-to-peer learning is not always structured or balanced.

In many learning groups, one person may dominate the discussion while others stay passive. Some learners hesitate to ask simple questions because they fear judgment.

Others may accept unclear or incorrect explanations without noticing the mistake. In addition, every learner has a different way of understanding and processing information. Some people learn better through examples, others through visual explanations, step-by-step guidance, discussion, practice, or repetition. Therefore, a single explanation method may not be effective for everyone in the group. As a result, the group can lose focus and finish the session without clear learning outcomes.

This problem is not limited to schools. It can happen in universities, online communities, workshops, training programs, study groups, and self-learning environments.

Main challenge: How can Generative AI act as a mediator, teammate, tutor, and facilitator to make learning more structured, inclusive, interactive, personalized, and effective?

GOALS:

This project aims to develop an innovative Generative AI-powered learning companion that supports both individual and group learning. The solution should help learners understand concepts, collaborate better, ask better questions, receive feedback, and revise more effectively.

1. Smart Peer Learning Support

Design an AI assistant that supports learners during study sessions. The AI should not only answer questions, but also guide the learning process. The system can:

- explain difficult concepts simply;
- ask guiding questions;
- propose hints before giving answers;
- detect confusion;
- suggest the next learning step.

2. Education and Accessible Learning

The solution should make learning more accessible by supporting different levels, languages, and learning styles. Since every learner understands information differently, the AI should first perform a diagnostic step to identify the method that fits each person best.

This diagnostic can be done through short questions, small exercises, previous answers, and interaction patterns. Based on the learner's responses, the AI can detect whether the person learns better through visual explanations, simple analogies, step-by-step reasoning, practical examples, repetition, or direct problem-solving. The AI can then adapt its explanations and activities to match the learner's preferred style and current level.

The AI can provide:

- simple explanations for beginners;
- step-by-step explanations for complex topics;
- examples from daily life;
- exercises adapted to the learner's level;
- personalized learning methods based on a diagnostic phase;
- multilingual support in Arabic, Darija, French, and English.

3. Open Innovation: Think Beyond the Tracks

Teams are encouraged to go beyond a simple chatbot and build a real learning companion. Possible extensions include:

- gamified learning progress.
- automatic revision notes.
- teacher dashboard.
- voice-based interaction.
- collaborative quizzes.
- learning analytics.

4. Entrepreneurship Angle

Think beyond the prototype: who would realistically use and sustain this companion (a school, a training center, a self-learning community), and what would it take to keep it running beyond the hackathon?

Teams must include a short business-model note: target users, value proposition, adoption path, and how the

Scanné avec CamScanner

solution could be sustained beyond the hackathon (sustainability/adoption plan).

CHALLENGE DETAILS:

A. Platform and Technical Constraints

This challenge is built on a real, operational AI inference service rather than a simulated environment. A working, end-to-end application is required: a mock-up, a notebook, or isolated API calls do not constitute a complete submission.

Two Categories of Language Models

The platform gives access to two categories of language models, and teams are expected to combine them deliberately:

- **Compact, fast models** — suited to short, well-defined tasks such as classification, intent detection, extraction, formatting, and short summaries.
- **Larger, more capable models** — suited to complex reasoning, multi-source synthesis, planning, and final polished writing, at the cost of noticeably higher response time and scarcer availability.

Sending every request to the larger model "to be safe" produces a slow, inefficient application. Using only the compact model puts a ceiling on quality. The ingenuity expected from teams lies in the combination: giving the compact model everything it can handle correctly, and reserving the larger model for steps that genuinely need it. Which category handles which step must be a documented, deliberate choice.

Operational Constraints

- Responses can take several seconds (more for the larger model category) since inference runs without graphics accelerators. This latency is a design budget to allocate, not an incident to report.
- Responses arrive as a whole rather than streamed word by word. The interface must give the user a sense of progress while waiting — loading indication, staged feedback, or a usable partial result — rather than a frozen screen.
- Each team's access is limited and shared. Applications must favour fewer, well-built requests over long chains of small calls, and must avoid regenerating an answer already obtained for an equivalent request.
- When the usage limit or the larger-model category becomes temporarily unavailable, the application must keep behaving correctly: queue the request, retry as guided, or fall back to the faster model category — and let the user know what is happening rather than failing silently.
- A document-retrieval capability (searchable knowledge base) may be used to narrow down the information given to a model before it generates an answer, improving relevance and reducing load.
- No custom training or fine-tuning is available. All intelligence must come from prompt design, the context supplied, the choice of which model category handles a step, and the overall application architecture.

Two-Phase Evaluation Process

The challenge will be conducted in two phases:

Phase 1 — Preselection

The first phase is a preselection phase for the top teams. No presentation is required during this phase. Teams must submit:

- a technical report (including the short business-model note);
- a GitHub repository link;
- a demonstration / simulation of the proposed solution.

Deadline: 10 November

The submitted report, GitHub repository, and demonstration / simulation will be used to select the teams that proceed to the final phase.

Phase 2 — Final Phase

The selected teams from Phase 1 will proceed to the final phase, which includes:

- the final presentation;
- the previous or a newer version of the video demonstration, not exceeding 1.5 minutes.

Final phase duration: 3 days after the Phase 1 selection

B. INSTRUCTIONS & DELIVERABLES

I. Final Presentation Guideline — Final Phase Format (selected teams only)

- Prepare a short presentation explaining the problem, solution, prototype, and impact.
- The pitch should be clear, direct, and easy to understand.

***Content to include*:**

- **Introduce your team members:** mention roles and areas of expertise.
- **Problem Definition:** explain clearly the learning problem your project addresses.
- **Proposed Solution:** present your GenAI learning companion and its main features.
- **Development Methodology:** briefly describe the AI model, RAG approach, tools, and platform used.
- **Implementation & Impact Strategy:** outline your prototype, user flow, testing plan, and expected results.
- **Business Model:** target users, value proposition, adoption path, and sustainability plan beyond the hackathon.
- Also describe how your application combines the two model categories, and how it behaves when usage is limited.

Scanné avec CamScanner

II. Demonstration Video

A short video should demonstrate the solution in action. It can show:

- the main problem or use case;
- the key features of the solution;
- how users interact with the system;
- the role of the AI or main technology used;
- the final output, result, or user benefit;
- the waiting or fallback behaviour, if relevant to your solution.

III. GitHub Repository & Technical Report

The GitHub repository should include:

- source code;
- setup instructions;
- README file;
- prototype screenshots if available;
- technical explanation of the GenAI module;
- a configuration template that does not expose any access credentials, if applicable.

The technical report should not exceed 6 pages and should clearly explain:

- project objective;
- system architecture;
- AI model or API used;
- RAG workflow if implemented;
- privacy and ethical safeguards;
- testing strategy and results;
- how the application handles limited capacity or temporary unavailability, if relevant.
- **business-model note:** target users, value proposition, adoption path, and how the solution could be sustained beyond the hackathon.

C. EVALUATION GRID

Projects will be evaluated according to clear and balanced criteria:

- **Innovation:** originality of the idea, creativity of the approach, and added value compared to existing solutions.
- **Technical Feasibility:** realism of the proposed solution, quality of the prototype, and clarity of the technical implementation.
- **Relevance:** alignment with the main challenge and ability to address a real user need.
- **Impact:** usefulness, accessibility, and potential benefits for the target users.
- **Entrepreneurship & Business Viability:** realistic target users, value proposition, a clear adoption path, and sustainability beyond the hackathon.
- **Pitch and Demo:** clarity of the presentation, quality of communication, and effectiveness of the demo.

Note: Technical Feasibility also takes into account how thoughtfully the two model categories are combined, how efficiently the service is used, and how the application behaves under limited capacity.

D. BONUS POINTS

Teams can earn extra points for going further in impact, usability, and technical quality.

- **Real User Testing:** test the prototype with learners, students, teachers, or training groups.
- **Multilingual Support:** include Arabic, Darija, French, or English interactions.

E. RULES

Team Composition:

- Each team may include up to 3 people.
- The pitch should be delivered by two team members.
- Teams are encouraged (not required) to have one member cover the entrepreneurship / business angle, in addition to the technical roles (AI, backend, frontend, design, documentation, pitch).

Ethics and Privacy:

- Do not store sensitive personal data.
- Use anonymous or synthetic testing data.
- Avoid collecting names, photos, voices, addresses, or private information.
- The AI must not discriminate by gender, region, language level, or background.
- The AI should support learning, not replace teachers or mentors.

Platform Usage:

- Access credentials are personal to the team and must not be shared, or exposed in code repositories, logs, screenshots, or videos.
- Attempting to bypass usage limits through additional accounts or replacement services during the demonstration is not permitted. Using an additional external service alongside the provided platform is acceptable if clearly declared.
- Only public, fictional, synthetic, or anonymised data may be sent to the service.
- Normal latency and occasional queueing are an expected part of the exercise, not a malfunction; persistent errors should be reported through the support channel provided by the organisers.
- Teams retain full ownership of their code and their work.

Pitching Format & Duration (Final Phase):

- **Presentation:** maximum 4 minutes.
- **Demo video:** maximum 1 minute and 30 seconds.
- **Q&A:** 2 minutes.

Scanné avec CamScanner